Functional Programming – Introduction

James Gain
Department of Computer Science
University of Cape Town
jgain@cs.uct.ac.za





Course Structure

- Introduction [2 lectures, JG]
  - The what and why of functional programming
- Supporting theory [3 lectures, JG]
  - Lazy evaluation, Lambda calculus
- Haskell [10 lecture, CM]
  - Core language features and coding examples
- Other Programming Paradigms [5 lectures, JG]
  - LLM-based code generation, logic programming, quantum programming

JG = James Gain, CM = Cael Marquard

Evaluation

CS3

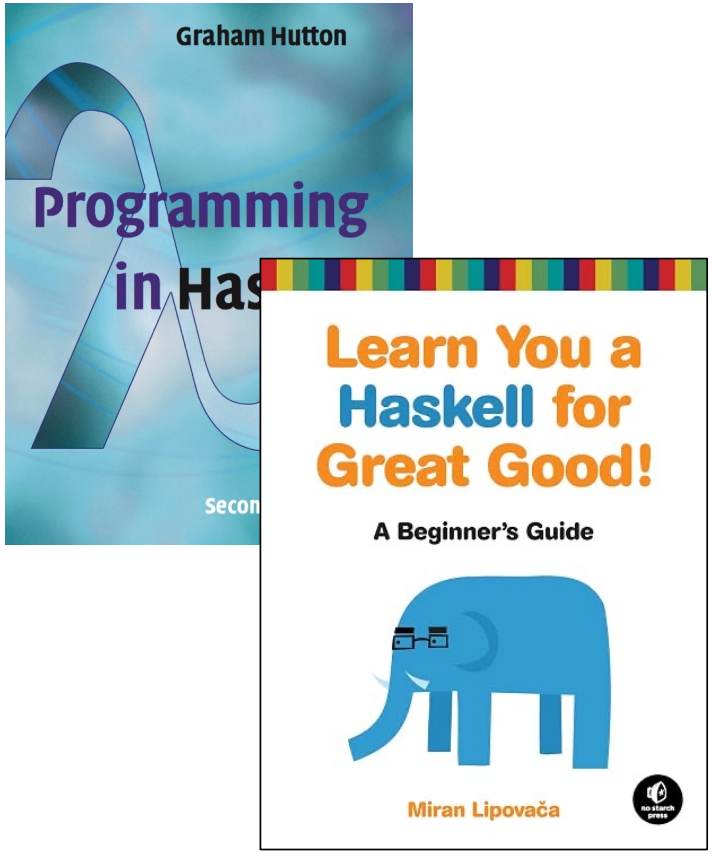
- Test
  - Part of Test 2
- Closed Prac
  - Mon 19 Oct & Thurs 22 Oct
- Exam
  - 10 November
  - 50% of $2\frac{1}{2}$ hour paper 1
- DP as per notes to students

Hons

- Test
  - None
- Closed Prac (25%)
  - Mon 26 Oct
- Exam (75%)
  - 10 November
  - $1\frac{1}{2}$ hour paper
- No DP requirement

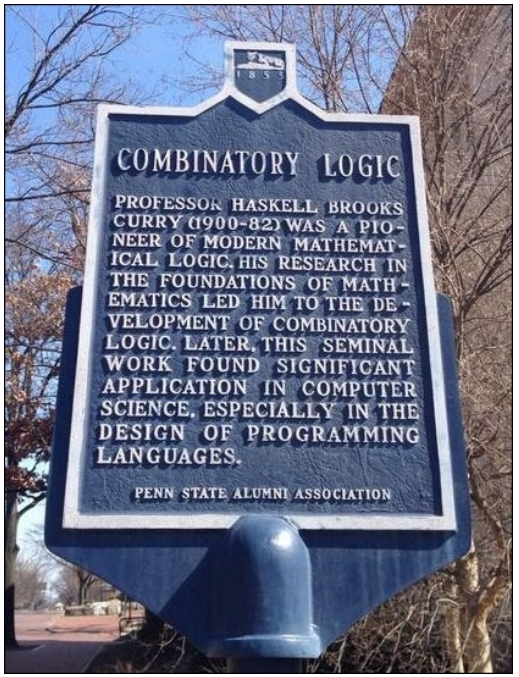
Readings

- Recommended textbook
  - Hutton, G. (2016) Programming in Haskell (2nd Edition). Cambridge University Press.
- Free option
  - Lipovaca, M. (2011) Learn you a Haskell for Great Good!: A Beginner's Guide
- Additional readings
  - Provided progressively under Readings tab on Amathuba



Haskell

- A statically-typed lazily-evaluated pure Functional Programming language
  - Used in this course
  - Named after Haskell Curry, a mathematician and computer scientist known for combinatory logic
  - Starting point: https://www.haskell.org
- Glasgow Haskell Compiler (GHC)
  - Standard compiler and interactive interpreter
  - Platform: https://www.haskell.org/platform/
  - EclipseFP plugin: eclipsefp.github.io/

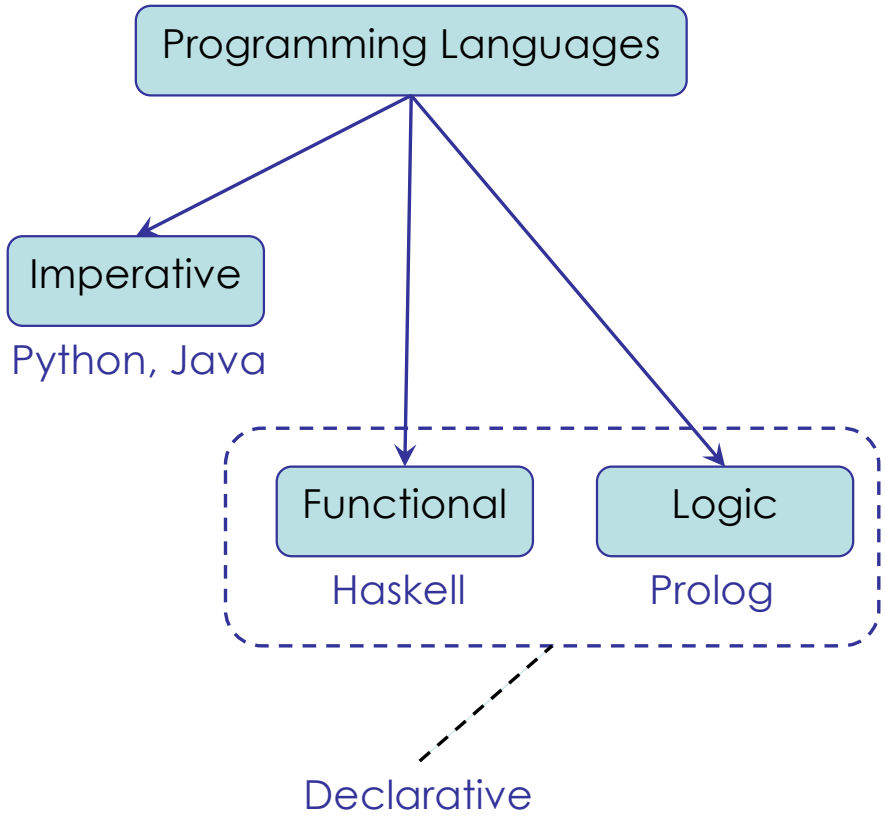


What is Functional Programming?

- A form of declarative programming by composing and applying mathematical functions
- Features:
  - Immutable – once created data cannot be modified
  - Pure – deterministically produce the same output given the same input without side effects
  - Functions as First-class citizens – functions can be passed as parameters, returned as values, and stored
- Not all functional programming languages obey all these features
- Some non-functional programming languages borrow elements

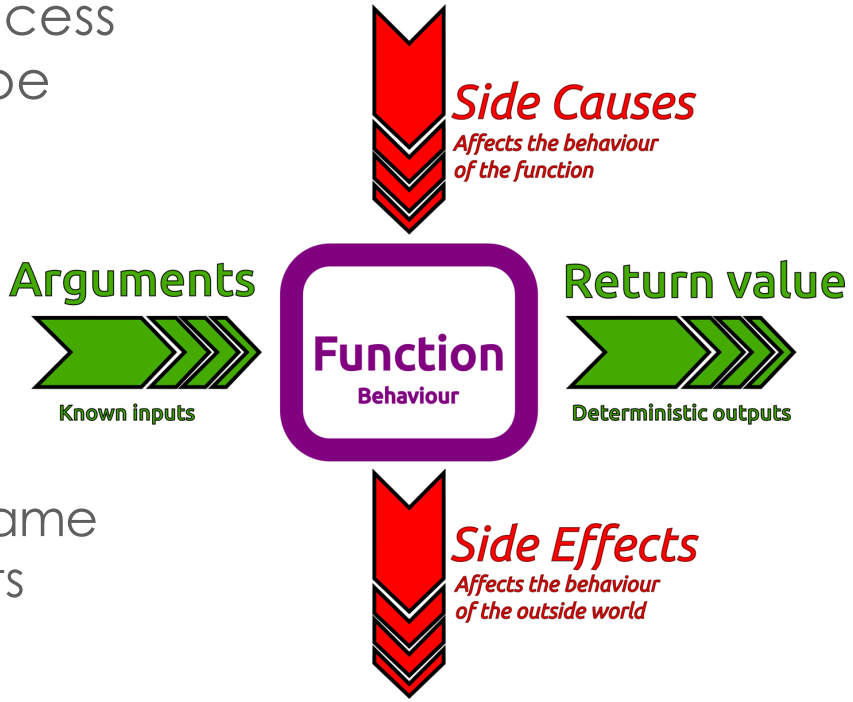
Declarative Programming

- 'What' rather than 'How'
  - Imperative is a step-by-step cookbook approach
  - Declarative describes the logic and is goal oriented
- Higher-level of abstraction
- State and control flow are managed by the system



Side Effects

- Imperative programs allow functions to access and change things outside their local scope
  - Modify global variables
  - Mutate function arguments
  - Input/output
  - Network requests
- Nondeterminism is a consequence
  - Can't guarantee that a function with the same inputs will always produce the same outputs
  - Hard to debug and reason about
  - But sometimes necessary
- Solved by immutable data and pure functions



Example: Side Effects

Imperative Programming

```
global even = True
def incr(value):
    global even
    even = not even
    if even:
        return value+1
    else:
        return value+2
print(incr(6))
print(incr(6))
```

Result:

```
8
7
```

Functional Programming

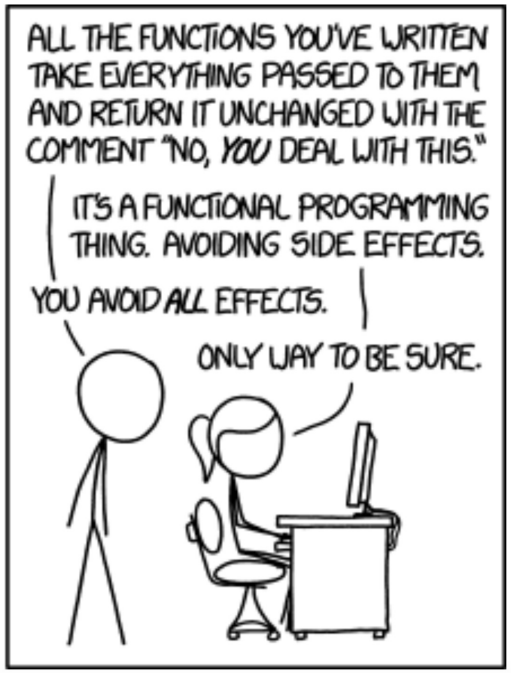
```
FUNC incr(INT value): INT
    RETURN value+1
print(incr(6))
print(incr(6))
```

Result:

```
7
7
```

Immutable Data

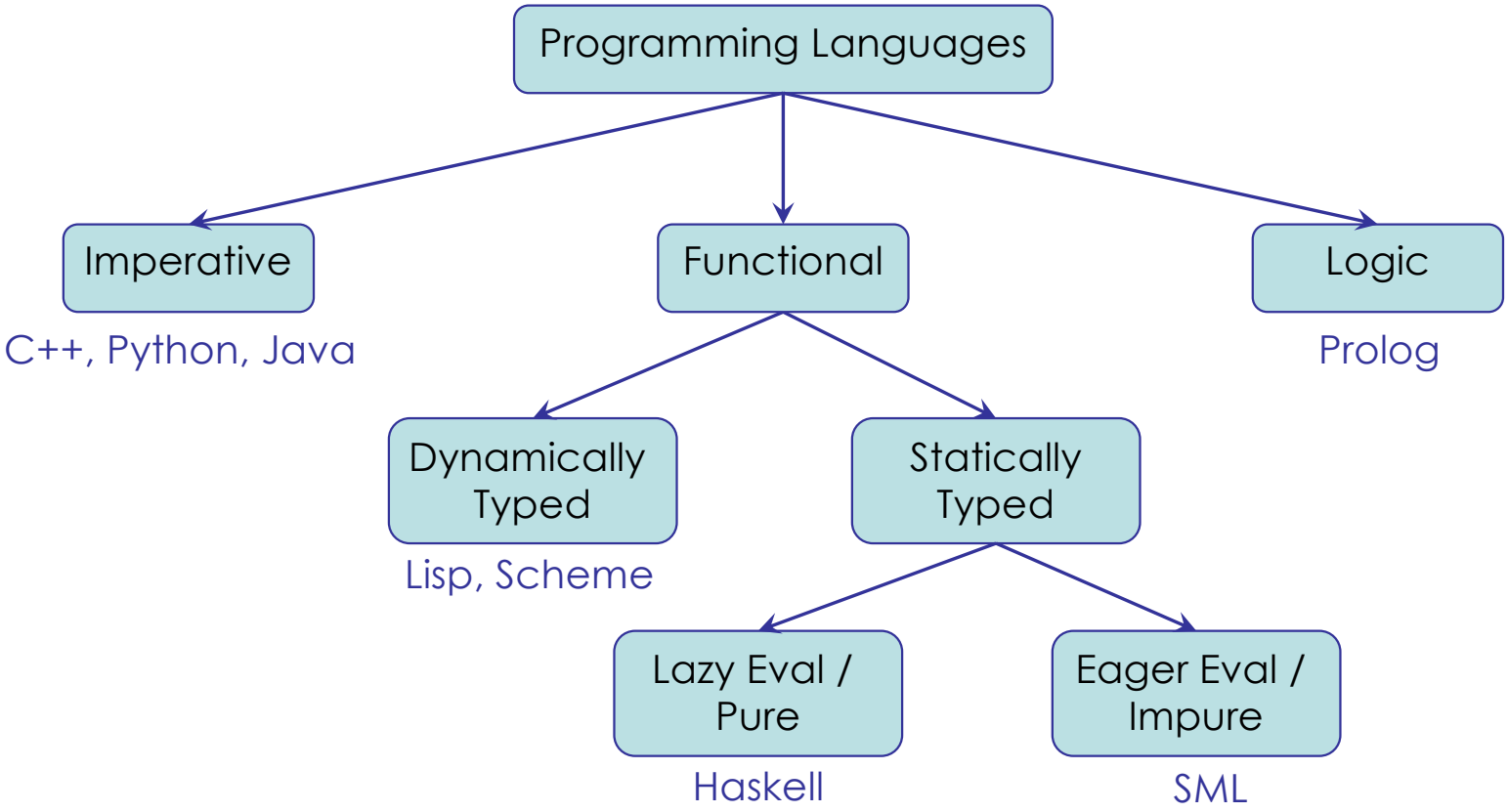
- Bind data to a name during initialization, afterwards it cannot be changed or overwritten
  - `A = [1,2,3,4]` ✓
    `A[0] = 0` ✗
  - Initializing assignment allowed
  - But mutating assignment disallowed
  - Instead of changing in place, FP creates new values and data structures attached to new names



Pure Functions

- Behave like deterministic mathematical functions:
  - Same input always yields the same output
  - No side effects
- Enables referential transparency
  - An expression (e.g., name or function) can always be replaced by its evaluated value without affecting program behaviour
- Hard to enforce in practice
  - Non-deterministic functions (`time.now()` and `random.rand()`) are opaque
  - Input from and output to an external source are opaque

Where Haskell fits in the language tree



Reminder: Typing

Dynamic

- Data types inferred automatically from values
- E.g., pie = 3.14
- Established at run time

(+) Faster to code, greater flexibility

(-) Possible run-time errors, potential performance cost

Static

- Data types explicitly declared in code
- E.g., float pie = 3.14
- Established at compile time

(+) Fewer bugs, better compiler optimization

(-) Slower to code, more verbose

Functional Programming Style

- No mutating assignment
  - Instead create new allocations
- No loops
  - Instead use function recursion
- But can lead to memory issues on both heap and stack
- Data-flow oriented
  - Design around composing simple functions together with data piped through them

```
-- Apply the square fn
-- to a list using map

square :: Int -> Int
square x = x * x

-- Result: [1, 4, 9, 16]
map square [1, 2, 3, 4]
```

Benefit: Parallelism

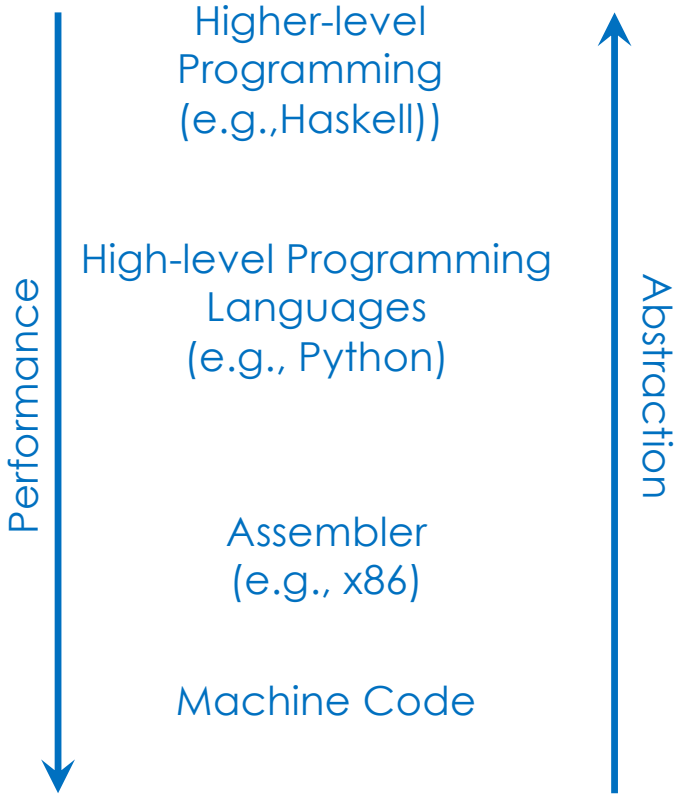
- Most imperative languages originated in the 80s and 90s
  - C++ (1983), Python (1991), Java (1995) – before the multi-core architecture era
  - Handle parallelism through threads with shared state controlled by locks, semaphores and atomics
  - Proving correctness with side effects is a nightmare
- Functional programming side-steps "writing while reading"
  - Because of immutability data can't be written to once created
  - Because of pure functions there are no side effects
  - Excellent fit for data stream parallelism

Benefit: Code Clarity

- Abstracts away a lot of execution detail
  - No worrying about memory locations, registers states, pointer manipulation
  - Native support for high-level operation (e.g., map, filter, reduce)
- Makes it easier to code
  - Can test (and prove correctness) of individual pure functions in isolation
  - More unit testing, less integration testing
  - Once you know the formalism, easier to read

Costs: Overheads

- Computation overheads
  - The higher the level of abstraction the less fine control
  - Functional can win from static typing and reordering of operations during compilation and lazy evaluation during execution
  - But imperative wins overall through low-level optimization
- Memory overheads
  - Lazy evaluation and recursion can impose stack and heap costs
  - More on this in the next lecture



Use in Industry

- FinTech
  - For type safety, predictable performance
- Integrated circuit design
  - For recursion
- Telecoms & messaging
  - For concurrency, scalability
- Natural language processing
  - For functional syntax



Best of Both Worlds?

- Why not add functional programming constructs to an imperative language?
  - This is the route taken by C++ (lambda expressions), Python (map), Java (util.stream)
- But …
  - Sprinkling `const` dust does not magically remove side effects
  - Functional is immutable by default and requires no thought
  - Imperative is mutable by default and requires extreme discipline



A jack of all trades is a master of none

Example: The Countdown Problem

- Using the numbers:

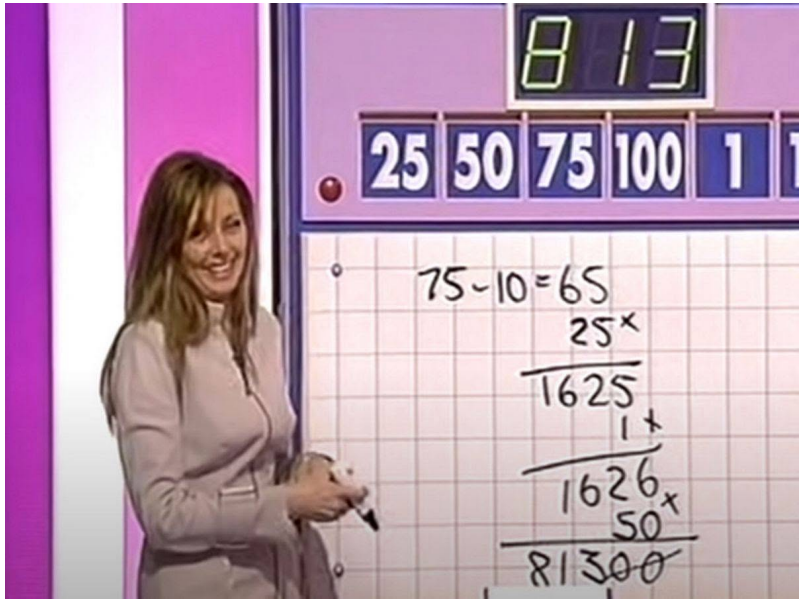
  `1` `3` `7` `10` `25` `50`

- And the arithmetic operators:

  `+` `-` `*` `÷`

- Construct an expression whose value is: `765`

- Rules:
  - Only use source numbers once
  - All numbers, including intermediates must be positive naturals



Inspired by the 'Countdown' British quiz TV show

Countdown: Solution

`(25-10) * (50+1)` = `765`

- There are 780 solutions in this example
- Changing the target to `831` leads to 0 solutions
- Brute-force solution
  - Create a list of lists with all possible permutations
  - Large search space with many inadmissible partial results
- E.g., [1, 3, 7, 10, 25, 20] 765 has ≈33 million expressions of which only ≈5 million are valid

Solution Performance

- System: 2.8 GHz Core 2 Duo, 4GB RAM
- Haskell Compiler: GHC version 7.10.2

	Brute-Force Functional	Optimized Functional
One Soln.	0.108s	0.014s
All Solns.	12.224s	1.312s

- Speedup achieved through lazy evaluation, search space pruning
- Hand-tuned Imperative will be faster but much more difficult to code, debug, and optimize
- Revisited once you have Haskell experience

Summary

- Functional programming is a form of declarative programming that offers immutable data handling and pure first-class function treatment
- Well suited to parallelism and high-level problem solving
- Comes at the potential cost of memory and performance overheads